

Persistência e Manipulação de Dados

Integração com Banco de Dados via JDBC

 Programação Web I



Objetivos da Aula

1. Revisitar conceitos básicos de banco de dados e SQL;
2. Compreender o conceito da API JDBC para integração de banco de dados em aplicações Java;
3. Utilizar a API JDBC para acessar e manipular dados em servidores *back-end* com Java;
4. Discutir os principais problemas e dificuldades na integração de Banco de Dados via JDBC puro.



Revisão sobre banco de dados e SQL

- **Banco de Dados:** coleção de dados estruturados e inter-relacionados que contém informações relevantes a um domínio;
- **SGBD: O Sistema Gerenciador de Banco de Dados**, corresponde a um **conjunto de programas** especificamente para facilitar a **criação e manutenção** de um conjunto de bancos de dados;
- A maior parte dos programas e aplicações de software utiliza banco de dados para **persistência de dados**;
- Com a evolução web, houve um **aumento significativo** na quantidade de aplicações de uso geral que provêem **interfaces para acesso e manipulação de dados**.

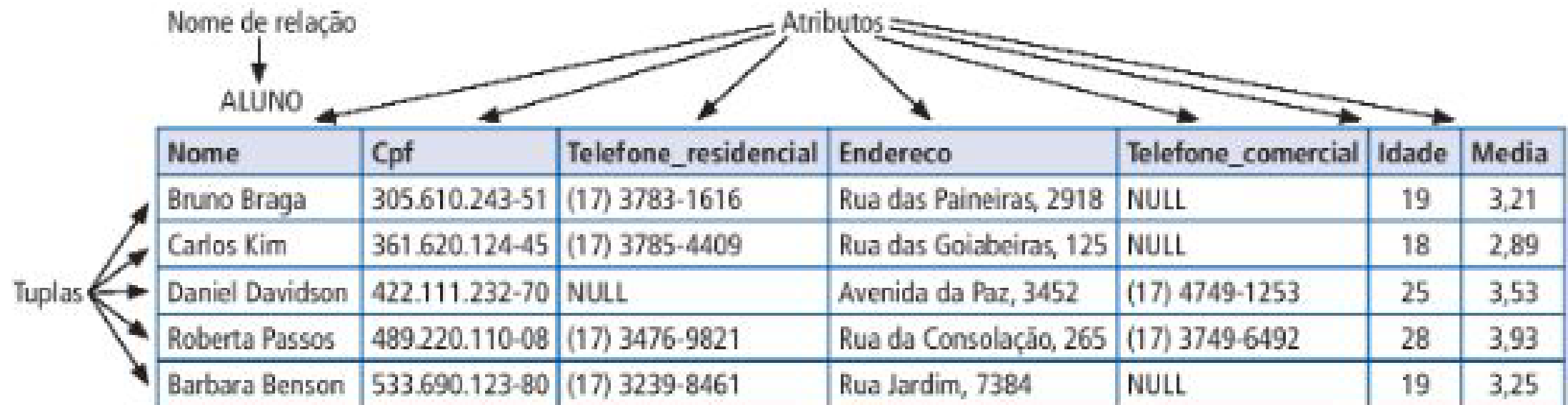


Revisão sobre banco de dados e SQL

- **Modelo Relacional:** representa o banco de dados como uma coleção de **relações**. Cada relação é semelhante a uma **tabela de valores**;
- Cada **linha (tupla)** na tabela, representa uma coleção de valores de dados relacionados, constituindo um **registro** dessa relação;
- Cada coluna da tabela é chamada de **atributo** da relação.

Revisão sobre banco de dados e SQL

Modelo Relacional





Revisão sobre banco de dados e SQL

- A principal linguagem de consulta e manipulação de dados em bancos de dados relacionais é a **SQL** (*Structured Query Language*);
- Subconjuntos de comandos SQL:
 - **DDL** (*Data Definition Language*): para definição de estruturas de dados (ex: `CREATE`, `ALTER`, `DROP`);
 - **DML** (*Data Manipulation Language*): para manipulação de dados (ex: `SELECT`, `INSERT`, `UPDATE`, `DELETE`);
 - **DCL** (*Data Control Language*): para controle de acesso e permissões (ex: `GRANT`, `REVOKE`).



Revisão sobre banco de dados e SQL

DDL

- **CREATE** : para criar uma nova tabela, banco de dados ou outro objeto;

```
CREATE TABLE aluno (  
    id INT PRIMARY KEY,  
    nome VARCHAR(100),  
    email VARCHAR(100),  
    cpf VARCHAR(11)  
);
```

- **ALTER** : para modificar a estrutura de uma tabela existente;

```
ALTER TABLE aluno ADD COLUMN idade INT;
```



Revisão sobre banco de dados e SQL

DML

- Nosso principal objetivo será aprender a **integrar comandos de manipulação de dados** em SQL dentro de nossas aplicações;
- Alguns dos comandos mais comuns da DML são:
 - **SELECT** : para recuperar dados de uma tabela;
 - **INSERT** : para adicionar novos registros em uma tabela;
 - **UPDATE** : para modificar dados existentes em uma tabela;
 - **DELETE** : para remover registros de uma tabela.



Revisão sobre banco de dados e SQL

DML

- **SELECT** :
 - Segue uma estrutura básica `SELECT <colunas> FROM <tabela> WHERE <condição>;`
 - Exemplo:

```
SELECT nome, email FROM usuarios WHERE idade > 18;
```



Revisão sobre banco de dados e SQL

DML

- **INSERT :**
 - Segue a estrutura `INSERT INTO <tabela> (<colunas>) VALUES (<valores>);`
 - Exemplo:

```
INSERT INTO aluno (id, nome, email, cpf)
VALUES (1, 'João Silva', 'joao.silva@example.com', '12345678901');
```



Revisão sobre banco de dados e SQL

DML

- **UPDATE :**
 - Segue a estrutura `UPDATE <tabela> SET <coluna>=<valor> WHERE <condição>;`
 - Exemplo:

```
UPDATE aluno SET email = 'joao.updated@example.com' WHERE id = 1;
```



Revisão sobre banco de dados e SQL

DML

- DELETE :
 - Segue a estrutura `DELETE FROM <tabela> WHERE <condição> ;`
 - Exemplo:

```
DELETE FROM aluno WHERE id = 1;
```

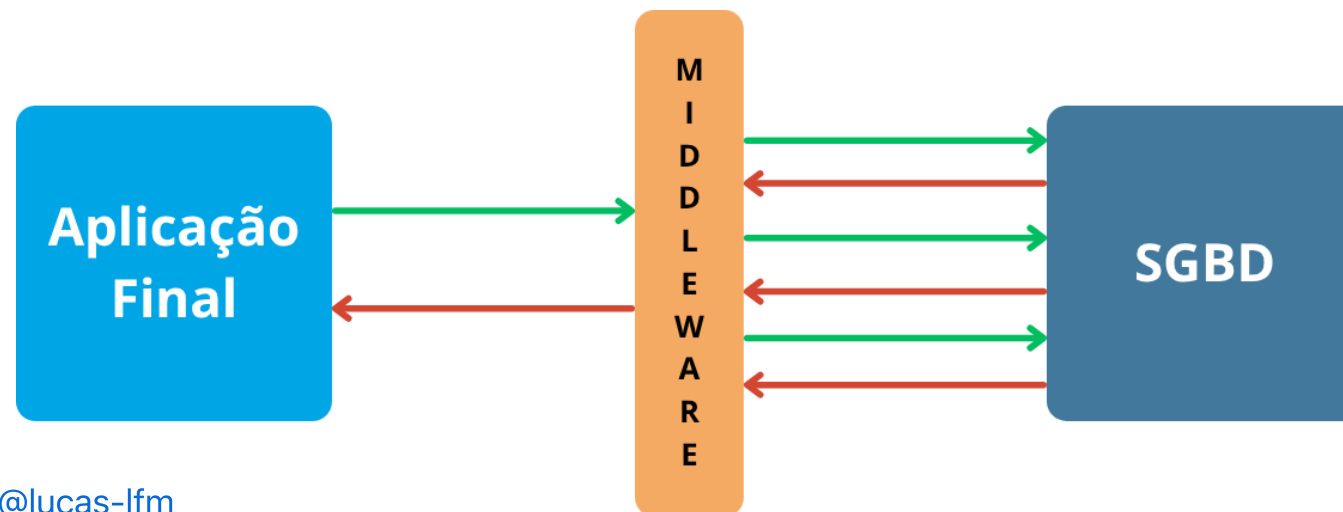
Integração de Banco de Dados em Aplicações

- A **maior parte do acesso a banco de dados** em aplicações práticas é feita por meio de programas de software, desenvolvidos em **linguagens de programação de uso geral**;
- Para isso, é necessário utilizar **bibliotecas ou APIs de acesso a banco de dados**, que permitem a integração de comandos SQL dentro do código da aplicação;
- A API padrão e amplamente utilizada para integração de banco de dados em Java é a **JDBC** (*Java Database Connectivity*).

Integração de Banco de Dados em Aplicações

Integração via bibliotecas e APIs como Middleware

- A principal técnica de integração consiste em uma camada de **middleware** entre a aplicação Java e o SGBD.



Integração de Banco de Dados em Aplicações

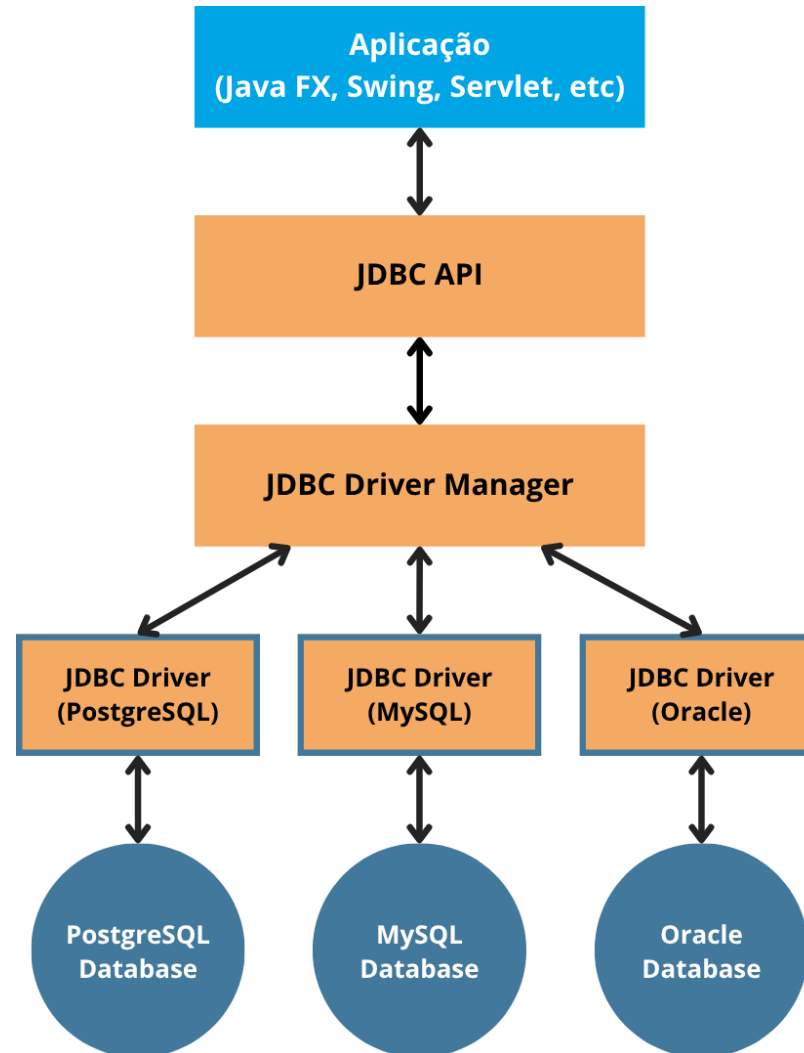
Integração via bibliotecas e APIs como Middleware

- A camada de middleware é responsável por:
 - **Traduzir as chamadas** de função/método da aplicação em comandos SQL compreensíveis pelo SGBD;
 - **Gerenciar a conexão com o banco de dados**, incluindo autenticação, manutenção de sessões e tratamento de erros;
 - Fornecer uma **interface de programação consistente** para a aplicação, **independente do SGBD** subjacente.

JDBC - Java Database Connectivity

- JDBC é uma **API padrão da linguagem Java** para **acesso a bancos de dados relacionais**;
- Fornece um conjunto de **interfaces e classes** para:
 - Estabelecer conexões com bancos de dados;
 - Executar comandos SQL;
 - Processar os resultados retornados pelo banco de dados.
- Um **Driver JDBC** é necessário para cada tipo de SGBD, atuando como um **adaptador**.

JDBC - Java Database Connectivity



JDBC - Java Database Connectivity

- Para utilizar JDBC, é necessário seguir os seguintes passos:
 - **Importar as classes JDBC** necessárias para o programa;
Pacote principal: `java.sql.*`
 - **Carregar o driver JDBC** para o SGBD específico;
 - **Estabelecer uma conexão** com o banco de dados usando a URL, nome de usuário e senha;
 - **Criar um objeto Statement** para enviar comandos SQL ao banco de dados;
 - **Executar comandos SQL** usando o Statement e processar os resultados;
 - **Fechar a conexão** e liberar recursos.

Driver JDBC

- O **Driver JDBC** é um componente de software que implementa a API JDBC para um SGBD específico;
- Ele atua como um **adaptador** entre a aplicação Java e o SGBD, traduzindo as chamadas JDBC em comandos SQL compreensíveis pelo SGBD;
- Para obter um driver JDBC, é necessário baixar o **JAR do driver** correspondente ao SGBD que você está utilizando.
 - Por exemplo: **MySQL Connector/J** para MySQL, **PostgreSQL JDBC Driver** para PostgreSQL.

Driver JDBC

- Em projetos com **Maven**, o driver JDBC pode ser adicionado como uma dependência no arquivo `pom.xml` :

```
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <version>42.7.11</version>
</dependency>
```

- Em projetos sem gerenciador de dependências, o **JAR do driver** deve ser adicionado manualmente ao classpath do projeto.

Driver JDBC

Driver Manager

- Componente fundamental da API JDBC, responsável por **gerenciar os drivers JDBC** disponíveis e **estabelecer conexões** com o banco de dados;
- Mantém uma **lista de drivers registrados** e é utilizado para **obter conexões** com o banco de dados por meio do método `getConnection()`.

```
Connection conn = DriverManager.getConnection(  
    "jdbc:postgresql://localhost:5432/meubanco",  
    "usuario",  
    "senha"  
);
```

Estabelecendo a Conexão

- Após adicionar a dependência do driver JDBC, o próximo passo é **estabelecer a conexão** com o banco de dados:

```
import java.sql.Connection;
import java.sql.DriverManager;

public class Main {
    public static void main(String[] args) throws Exception {
        String url = "jdbc:postgresql://localhost:5432/meubanco";
        String usuario = "postgres";
        String senha = "123456";

        Connection conn = DriverManager.getConnection(url, usuario, senha);

        System.out.println("Conectado com sucesso!");
        conn.close();
    }
}
```

✓ Estabelecendo a Conexão

- A **string de conexão** (URL) passada para `getConnection()` segue um formato específico:
 - `jdbc:<subprotocolo>:<subnome>`
 - Exemplo para PostgreSQL: `jdbc:postgresql://localhost:5432/meubanco`
 - Exemplo para MySQL: `jdbc:mysql://localhost:3306/meubanco`
- Ao identificar o prefixo `jdbc:postgresql`, o Driver Manager sabe que deve usar o driver PostgreSQL para estabelecer a conexão.

Estabelecendo a Conexão

- A classe `Connection` implementa a interface `AutoCloseable`, o que permite o uso do **try-with-resources** para garantir que a conexão seja fechada automaticamente:

```
try (Connection conn = DriverManager.getConnection(url, usuario, senha)) {  
    System.out.println("Conectado com sucesso!");  
} catch (SQLException e) {  
    System.err.println("Erro ao conectar ao banco de dados:");  
    e.printStackTrace();  
}
```

Vamos optar pelo uso de **try-with-resources** para garantir o fechamento adequado da conexão em nossos exemplos práticos.



Estabelecendo a Conexão - Exemplificando

```
public class Main {  
    public static void main(String[] args) {  
        String url = "jdbc:postgresql://localhost:5432/meubanco";  
        String usuario = "postgres";  
        String senha = "123456";  
  
        try (Connection conn = DriverManager.getConnection(url, usuario, senha);  
            Statement stmt = conn.createStatement();  
            ResultSet rs = stmt.executeQuery("SELECT * FROM aluno")) {  
  
            while (rs.next()) {  
                String nome = rs.getString("nome");  
                String email = rs.getString("email");  
                System.out.println("Nome: " + nome + ", Email: " + email);  
            }  
        } catch (SQLException e) {  
            System.err.println("Erro ao conectar ao banco de dados:");  
            e.printStackTrace();  
        }  
    }  
}
```



Factory Method

 O que aconteceria se precisássemos chamar `DriverManager.getConnection()` em diversos pontos do código?

- O **Factory Method** é um **design pattern** que utilizamos para **centralizar a criação de objetos** e **ocultar a complexidade** da criação de instâncias;
- Em vez de espalhar a lógica de conexão em vários lugares, podemos criar um **método de fábrica** para centralizar a criação da conexão.



Factory Method

```
public class ConnectionFactory {  
  
    private static final String URL = "jdbc:postgresql://localhost:5432/loja";  
    private static final String USER = "postgres";  
    private static final String PASSWORD = "123456";  
  
    public Connection getConnection() {  
        try {  
            return DriverManager.getConnection(URL, USER, PASSWORD);  
        } catch (SQLException e) {  
            throw new RuntimeException("Erro ao conectar ao banco", e);  
        }  
    }  
}
```



Factory Method

- Agora podemos obter conexões de forma centralizada e consistente:

```
public class Main {  
    public static void main(String[] args) {  
        ConnectionFactory factory = new ConnectionFactory();  
  
        try (Connection conn = factory.getConnection();  
            Statement stmt = conn.createStatement();  
            ResultSet rs = stmt.executeQuery("SELECT * FROM aluno")) {  
  
            while (rs.next()) {  
                ...  
            }  
        } catch (SQLException e) {  
            System.err.println("Erro ao conectar ao banco de dados:");  
            e.printStackTrace();  
        }  
    }  
}
```

Execução de comandos SQL

- Para executar comandos SQL, utilizamos a interface `Statement` ou `PreparedStatement` :
 - `Statement` é utilizado para comandos SQL estáticos;
 - `PreparedStatement` é utilizado para comandos SQL dinâmicos, permitindo a utilização de parâmetros.
- Temos basicamente 2 tipos de comandos SQL para a JDBC:
 - **Consultas**: utilizam o método `executeQuery()`, que retorna um `ResultSet` com os resultados da consulta;
 - **Atualizações**: utilizam os métodos `executeUpdate()` ou `execute()`, que retornam o número de linhas afetadas ou um boolean indicando se o resultado é um `ResultSet`.

Execução de comandos SQL

- Consulta utilizando `Statement` :

```
try (Connection conn = factory.getConnection();
    Statement stmt = conn.createStatement();
    ResultSet rs = stmt.executeQuery("SELECT * FROM aluno")) {

    while (rs.next()) {
        String nome = rs.getString("nome");
        String email = rs.getString("email");
        System.out.println("Nome: " + nome + ", Email: " + email);
    }
}
```

- Perceba que o método `executeQuery()` é utilizado para consultas;
- O `ResultSet` retornado pode ser iterado para acessar os dados retornados pelo banco de dados.

Execução de comandos SQL

- Consulta utilizando PreparedStatement :

```
String sql = "SELECT * FROM aluno WHERE idade > ?";
try (Connection conn = factory.getConnection();
    PreparedStatement pstmt = conn.prepareStatement(sql)) {

    pstmt.setInt(1, 18); // Define o valor do parâmetro

    try (ResultSet rs = pstmt.executeQuery()) {
        while (rs.next()) {
            String nome = rs.getString("nome");
            String email = rs.getString("email");
            System.out.println("Nome: " + nome + ", Email: " + email);
        }
    }
}
```

Execução de comandos SQL

Vantagens do `PreparedStatement`:

- O método `prepareStatement` da classe `Connection` submete um comando SQL para compilação;
- Nesse ponto, **nenhum comando SQL foi executado** ainda;
- É necessário definir os valores dos parâmetros utilizando os métodos `setXXX()`, na ordem em que eles aparecem no comando SQL;
- Com `PreparedStatement` podemos evitar problemas de **SQL Injection**. Talvez essa seja a vantagem mais importante;

Execução de comandos SQL

Vantagens do PreparedStatement:

- Imagine que temos um comando SQL dinâmico, onde o valor de um parâmetro é fornecido pelo usuário:

```
String sql = "SELECT * FROM aluno WHERE nome = '" + nomeUsuario + "'";  
Statement stmt = conn.createStatement();  
ResultSet rs = stmt.executeQuery(sql);
```

- Se o usuário fornecer um valor malicioso, como `João' OR '1'='1'`, o comando SQL resultante seria:

```
SELECT * FROM aluno WHERE nome = 'João' OR '1'='1'
```

Execução de comandos SQL

Vantagens do PreparedStatement:

- Com o PreparedStatement, o comando SQL seria:

```
String sql = "SELECT * FROM aluno WHERE nome = ?";  
PreparedStatement pstmt = conn.prepareStatement(sql);  
pstmt.setString(1, nomeUsuario);  
ResultSet rs = pstmt.executeQuery();
```

- Nesse caso, o valor do parâmetro é tratado como um **valor literal** (escapado), e não como parte do comando SQL.

Execução de comandos SQL

Inserção, Atualização e Exclusão de Dados:

- Para comandos de manipulação de dados (DML), utilizamos os métodos `executeUpdate()` ou `execute()`. Exemplo de **atualização**:

```
String sql = "UPDATE aluno SET email = ? WHERE id = ?";
try (Connection conn = factory.getConnection();
    PreparedStatement pstmt = conn.prepareStatement(sql)) {

    pstmt.setString(1, "joao.silva@example.com");
    pstmt.setInt(2, 1);
    int rowsAffected = pstmt.executeUpdate();
    System.out.println("Linhas atualizadas: " + rowsAffected);
}
```

Execução de comandos SQL

Inserção, Atualização e Exclusão de Dados:

- Inserção:

```
String sql = "INSERT INTO aluno (id, nome, email) VALUES (?, ?, ?)";
try (Connection conn = factory.getConnection();
    PreparedStatement pstmt = conn.prepareStatement(sql)) {

    pstmt.setInt(1, 2);
    pstmt.setString(2, "Maria Silva");
    pstmt.setString(3, "maria.silva@example.com");
    int rowsAffected = pstmt.executeUpdate();
    System.out.println("Linhas inseridas: " + rowsAffected);
}
```

Execução de comandos SQL

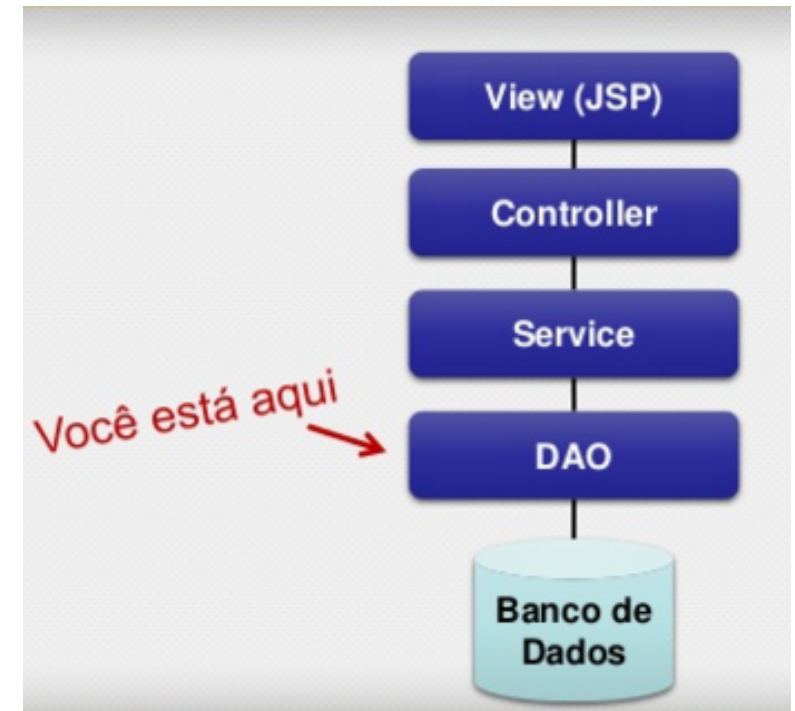
Inserção, Atualização e Exclusão de Dados:

- Exclusão:

```
String sql = "DELETE FROM aluno WHERE id = ?";  
try (Connection conn = factory.getConnection();  
    PreparedStatement pstmt = conn.prepareStatement(sql)) {  
  
    pstmt.setInt(1, 2);  
    int rowsAffected = pstmt.executeUpdate();  
    System.out.println("Linhas excluídas: " + rowsAffected);  
}
```

Padrão DAO - Data Access Object

- O **Padrão DAO** é um design pattern que tem como objetivo **separar a lógica de acesso a dados** da lógica de negócio da aplicação;
- Define uma **interface** para as operações de acesso a dados, e uma ou mais **implementações concretas**;
- As implementações do DAO são responsáveis por **interagir com o banco de dados** utilizando JDBC ou outra tecnologia de acesso a dados.



Padrão DAO - Data Access Object

Seção em Atualização...

HORA DE PRATICAR!

Requisitos para as práticas

- Ter o **JDK 21** ou superior instalado;
- Ter o **PostgreSQL** instalado e rodando localmente;
- Ter o **pgAdmin** instalado para gerenciar o banco de dados (geralmente instalado junto com o PostgreSQL);
- Pegar a **dependência do driver JDBC** para PostgreSQL e adicioná-la ao projeto (via Maven, `pom.xml`). Ou baixar o JAR do driver e adicioná-lo manualmente ao classpath do projeto.

Acesse <https://jdbc.postgresql.org/download/> para obter a versão mais recente do driver JDBC para PostgreSQL.

Prática 1 - Testando a Conexão

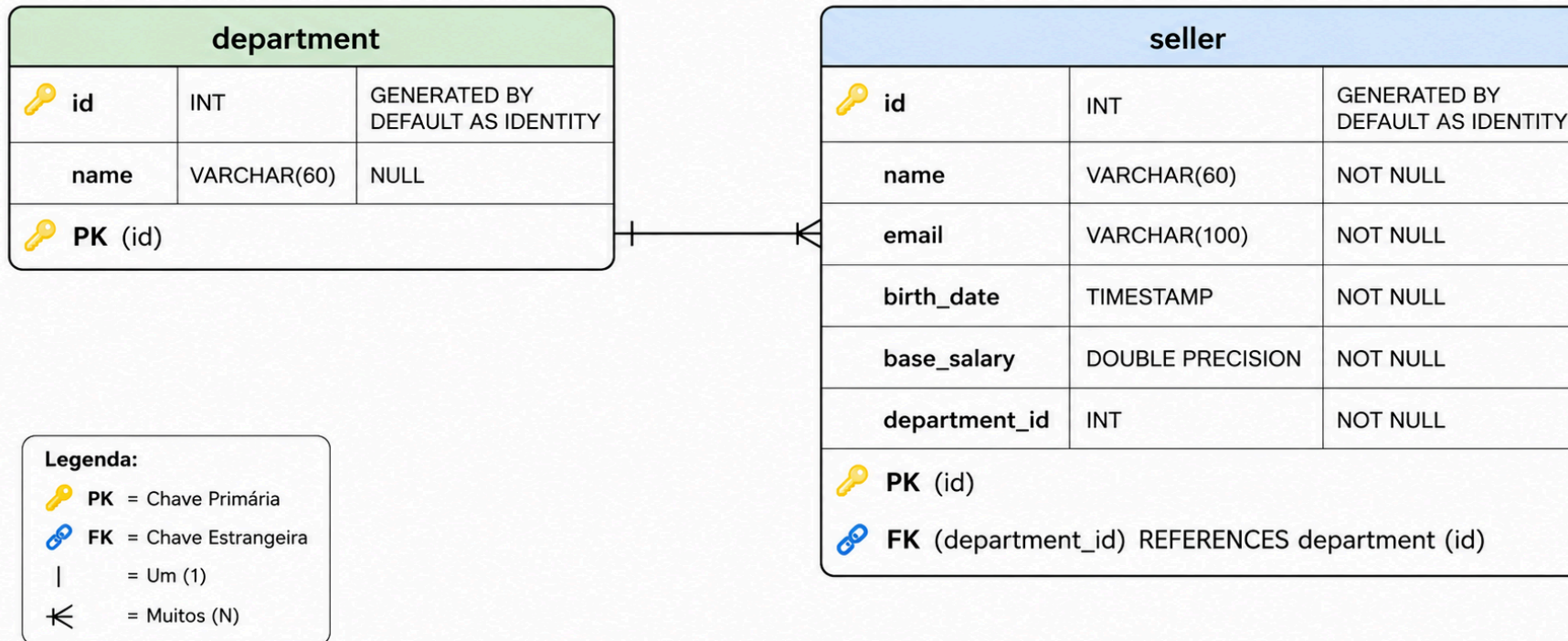
1. Crie um novo projeto Java, com Maven (sem nenhum archetype), e adicione a dependência do driver JDBC para o SGBD PostgreSQL no arquivo `pom.xml` :

```
<dependency>  
  <groupId>org.postgresql</groupId>  
  <artifactId>postgresql</artifactId>  
  <version>42.7.11</version>  
</dependency>
```

2. Abra o `pgAdmin` e crie um novo banco de dados chamado `loja_jdbc`.
3. Na `Query Tool` do banco `loja_jdbc`, execute o script SQL disponível no seguinte gist: <https://gist.github.com/lucas-lfm/cac01a20bd407271ac2b914276e26b65>

Prática 1 - Testando a Conexão

- O script SQL do gist anterior irá criar o seguinte modelo de dados:



Prática 1 - Testando a Conexão

4. No arquivo `src/main/java/Main.java`, teste a conexão com o banco de dados utilizando a classe `DriverManager` e imprima uma mensagem de sucesso ou erro:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class Main {
    public static void main(String[] args) {
        String url = "jdbc:postgresql://localhost:5432/loja_jdbc";
        String usuario = "postgres";
        String senha = "123456";

        try (Connection conn = DriverManager.getConnection(url, usuario, senha)) {
            System.out.println("Conectado com sucesso!");
        } catch (SQLException e) {
            System.err.println("Erro ao conectar ao banco de dados:");
            e.printStackTrace();
        }
    }
}
```

Prática 1 - Testando a Conexão

5. Execute o programa e verifique se a conexão foi estabelecida com sucesso.

a. No VS Code, clique com o botão direito no arquivo `Main.java` > Run Java

6. Caso haja algum erro, verifique a string de conexão, as credenciais e se o banco de dados está rodando corretamente.

Prática 2 - Consultando Dados

1. No arquivo `Main.java`, após estabelecer a conexão, crie um objeto `Statement` e execute uma consulta SQL para recuperar todos os vendedores da tabela `seller`:

```
try (Connection conn = DriverManager.getConnection(url, usuario, senha);
    Statement stmt = conn.createStatement();
    ResultSet rs = stmt.executeQuery("SELECT * FROM seller")) {

    while (rs.next()) {
        String nome = rs.getString("name");
        Double salario = rs.getDouble("base_salary");
        System.out.println("Vendedor: " + nome + ", Salário: " + salario);
    }

} catch (SQLException e) {
    System.err.println("Erro ao consultar o banco de dados:");
    e.printStackTrace();
}
```

Prática 2 - Consultando Dados

2. Execute o programa e verifique se os vendedores foram listados corretamente no console.
3. Modifique a consulta para recuperar apenas os vendedores com salário maior que 5000:

```
try (Connection conn = DriverManager.getConnection(url, usuario, senha);
    Statement stmt = conn.createStatement();
    ResultSet rs = stmt.executeQuery("SELECT * FROM seller WHERE base_salary > 5000")) {

    // Código para processar o ResultSet

} catch (SQLException e) {
    // Tratamento de erro
}
```

Prática 3 - Inserindo Dados

1. No arquivo `Main.java`, crie um objeto `PreparedStatement` para inserir um novo vendedor na tabela `seller`:

```
String sql = "INSERT INTO seller (name, email, birth_date, base_salary, department_id)
VALUES (?, ?, ?, ?, ?)";
try (Connection conn = DriverManager.getConnection(url, usuario, senha);
    PreparedStatement pstmt = conn.prepareStatement(sql)) {
    pstmt.setString(1, "Carlos Silva");
    pstmt.setString(2, "carlos.silva@email.com");
    pstmt.setDate(3, Date.valueOf("1985-06-15"));
    pstmt.setDouble(4, 6000.0);
    pstmt.setInt(5, 1);

    int rowsAffected = pstmt.executeUpdate();
    System.out.println("Vendedor inserido com sucesso! Linhas afetadas: " + rowsAffected);
} catch (SQLException e) {
    System.err.println("Erro ao inserir vendedor:");
    e.printStackTrace();
}
```

Prática 3 - Inserindo Dados

2. Execute o programa e vá ao `pgAdmin` para verificar se o vendedor "Carlos Silva" foi adicionado à tabela `seller`.
3. Modifique o código para inserir um novo departamento na tabela `department`:

```
String sql = "INSERT INTO department (name) VALUES (?)";
try (Connection conn = DriverManager.getConnection(url, usuario, senha);
    PreparedStatement pstmt = conn.prepareStatement(sql)) {
    pstmt.setString(1, "Recursos Humanos");
    int rowsAffected = pstmt.executeUpdate();
    System.out.println("Departamento inserido com sucesso! Linhas afetadas: " + rowsAffected);
} catch (SQLException e) {
    System.err.println("Erro ao inserir departamento:");
    e.printStackTrace();
}
```

Prática 4 - Atualizando e Excluindo Dados

1. No arquivo `Main.java`, crie um objeto `PreparedStatement` para atualizar o salário de um vendedor específico:

```
String sql = "UPDATE seller SET base_salary = ? WHERE name = ?";
try (Connection conn = DriverManager.getConnection(url, usuario, senha);
    PreparedStatement pstmt = conn.prepareStatement(sql)) {
    pstmt.setDouble(1, 6500.0);
    pstmt.setString(2, "Carlos Silva");

    int rowsAffected = pstmt.executeUpdate();
    System.out.println("Salário atualizado com sucesso! Linhas afetadas: " + rowsAffected);
} catch (SQLException e) {
    System.err.println("Erro ao atualizar salário:");
    e.printStackTrace();
}
```

Prática 4 - Atualizando e Excluindo Dados

2. Execute o programa e verifique se o salário do vendedor "Carlos Silva" foi atualizado no banco de dados.
3. Modifique o código para excluir um vendedor específico da tabela `seller`:

```
String sql = "DELETE FROM seller WHERE id = ?";
try (Connection conn = DriverManager.getConnection(url, usuario, senha);
    PreparedStatement pstmt = conn.prepareStatement(sql)) {
    pstmt.setInt(1, 3); // ID do vendedor a ser excluído
    int rowsAffected = pstmt.executeUpdate();
    System.out.println("Vendedor excluído com sucesso! Linhas afetadas: " + rowsAffected);
} catch (SQLException e) {
    System.err.println("Erro ao excluir vendedor:");
    e.printStackTrace();
}
```

Prática 5 - Manipulações Avançadas

1. Vamos realizar uma junção entre as tabelas `seller` e `department` para listar os vendedores junto com o nome do departamento:

```
String sql = "SELECT s.name AS seller_name, d.name AS department_name " +  
            "FROM seller s " +  
            "JOIN department d ON s.department_id = d.id";  
try (Connection conn = DriverManager.getConnection(url, usuario, senha);  
    Statement stmt = conn.createStatement();  
    ResultSet rs = stmt.executeQuery(sql)) {  
    while (rs.next()) {  
        String sellerName = rs.getString("seller_name");  
        String departmentName = rs.getString("department_name");  
        System.out.println("Vendedor: " + sellerName + ", Departamento: " + departmentName);  
    }  
} catch (SQLException e) {  
    System.err.println("Erro ao executar consulta com junção:");  
    e.printStackTrace();  
}
```

Prática 5 - Manipulações Avançadas

2. Atualize o código para calcular a média salarial dos vendedores por departamento:

```
String sql = "SELECT d.name AS department_name, AVG(s.base_salary) AS avg_salary " +  
            "FROM seller s " +  
            "JOIN department d ON s.department_id = d.id " +  
            "GROUP BY d.name";  
try (Connection conn = DriverManager.getConnection(url, usuario, senha);  
    Statement stmt = conn.createStatement();  
    ResultSet rs = stmt.executeQuery(sql)) {  
    while (rs.next()) {  
        String departmentName = rs.getString("department_name");  
        double avgSalary = rs.getDouble("avg_salary");  
        System.out.println("Departamento: " + departmentName + ", Salário Médio: " + avgSalary);  
    }  
} catch (SQLException e) {  
    System.err.println("Erro ao calcular média salarial:");  
    e.printStackTrace();  
}
```

Prática 5 - Manipulações Avançadas

3. Modifique o código para atualizar o salário de todos os vendedores de um departamento específico, aumentando em 10%:

```
String sql = "UPDATE seller SET base_salary = base_salary * 1.1 WHERE department_id = ?";
try (Connection conn = DriverManager.getConnection(url, usuario, senha);
    PreparedStatement pstmt = conn.prepareStatement(sql)) {
    pstmt.setInt(1, 2); // ID do departamento
    int rowsAffected = pstmt.executeUpdate();
    System.out.println("Salários atualizados com sucesso! Linhas afetadas: " + rowsAffected);
} catch (SQLException e) {
    System.err.println("Erro ao atualizar salários:");
    e.printStackTrace();
}
```

Prática 5 - Manipulações Avançadas

4. **Desafio:** Crie um método que receba o nome de um departamento e retorne a lista de vendedores desse departamento, utilizando `PreparedStatement` para evitar SQL Injection.
5. **Desafio:** Implemente um método para consultar os vendedores com idade acima de 30 anos, pertencentes a um departamento específico. Utilize `PreparedStatement` para passar o parâmetro do departamento e a idade.

Prática 6 - Padrão DAO e Camada de Serviço

1. Crie um novo projeto Java, com Maven, e adicione a dependência do driver JDBC para PostgreSQL no arquivo `pom.xml`.
2. Crie as seguintes classes de modelo (model):
 - a. `Department` com os atributos: `id`, `name`;
 - b. `Seller` com os atributos: `id`, `name`, `email`, `birthDate`, `baseSalary`, `departmentId`.
3. Crie uma interface `SellerDAO` com os seguintes métodos:
 - a. `List<Seller> findAll()`, `Seller findById(int id)`, `void insert(Seller seller)`, `void update(Seller seller)`, `void delete(int id)`.

Prática 6 - Padrão DAO e Camada de Serviço

4. Crie uma classe `ConnectionFactory` para centralizar a criação de conexões com o banco de dados, utilizando o padrão Factory Method.
5. Crie uma classe `SellerDAOImpl` que implemente a interface `SellerDAO`, utilizando JDBC para implementar os métodos de acesso a dados. Certifique-se de utilizar `PreparedStatement` para evitar SQL Injection e de fechar os recursos adequadamente.
6. Crie uma classe `SellerService` que utilize a `SellerDAO` para fornecer métodos de negócio relacionados aos vendedores.

Prática 6 - Padrão DAO e Camada de Serviço

7. Teste a implementação criando um `Main` para inserir, consultar, atualizar e excluir vendedores utilizando a camada de serviço (`SellerService`), que por sua vez utiliza a camada de acesso a dados (`SellerDAO`).
8. **Desafio:** Implemente um método na `SellerService` para calcular a média salarial dos vendedores por departamento, utilizando a `SellerDAO` para acessar os dados necessários.

Prática 6 - Padrão DAO e Camada de Serviço

9. Crie uma classe `DepartmentDAO` e `DepartmentDAOImpl` para acessar os dados dos departamentos, seguindo o mesmo padrão utilizado para os vendedores.
10. Implemente um método na `SellerService` para listar os vendedores junto com o nome do departamento, utilizando as DAOs de vendedor e departamento para acessar os dados.
11. **Desafio:** Implemente um método na `SellerService` para atualizar o salário de todos os vendedores de um departamento específico, aumentando em 10%, utilizando as DAOs para acessar e atualizar os dados.



Próximas Aulas

-